11. Presentación de Stack Tecnológico

Para desarrollar el Sistema de Control Transmetro se seleccionó un conjunto de tecnologías que permite construir una aplicación web segura, organizada, escalable y fácil de mantener, capaz de operar desde la PC de trabajo de cada estación y de crecer conforme se incorporen nuevas líneas y municipalidades. El stack tecnológico comprende la base de datos, la metodología de trabajo, el lenguaje de programación, el framework, el entorno de desarrollo integrado, la plataforma de despliegue y las herramientas de calidad que acompañan todo el ciclo de vida del proyecto, desde el análisis hasta la operación.

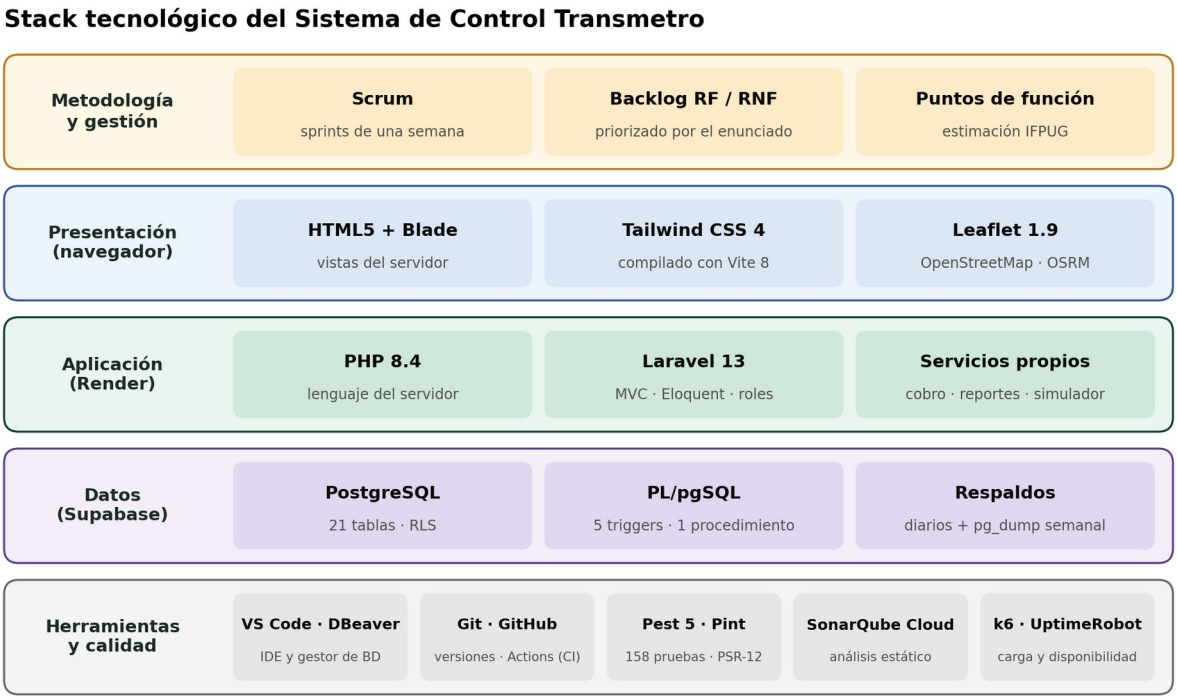


Figura 9. Stack tecnológico del Sistema de Control Transmetro.

Base de datos: PostgreSQL

Como sistema gestor de base de datos se utilizará **PostgreSQL**, un motor relacional de código abierto reconocido por su cumplimiento del estándar SQL, su manejo de transacciones y su capacidad para aplicar reglas de integridad dentro de la propia base de datos. PostgreSQL almacenará y administrará toda la información del sistema: municipalidades, líneas, horarios, estaciones, accesos, molinetes, parqueos, buses, viajes, personal, tarjetas, recargas, pagos, alertas, usuarios y bitácora de auditoría.

La base de datos está compuesta por 21 tablas relacionadas mediante llaves primarias de tipo UUID y llaves foráneas. Las tablas se organizan en seis grupos según su función:

- **Estructura territorial y rutas:** municipios, lineas, horarios_linea, estaciones y linea_estacion (orden de las paradas y distancia entre ellas).
- **Infraestructura de estaciones:** accesos, molinetes y parqueos.
- **Flota y operación:** buses, viajes, visitas_viaje y alertas_sistema.
- **Personal:** empleados, pilotos, seguridad y operadores.
- **Pagos:** tarjetas, recargas y transacciones_pago.
- **Seguridad y auditoría:** users y bitacora_auditoria.

Las reglas del enunciado se aplicarán directamente en la base de datos, de modo que ninguna operación pueda dejar la información en un estado inválido, aunque se intente modificarla fuera de la aplicación:

- Restricciones CHECK para valores válidos: recargas de Q5.00 a Q200.00, ocupación de 0 a 100 %, estados permitidos y métodos de pago sin efectivo.
- Trigger que impide que una estación quede con menos de dos accesos y otro que impide que un acceso quede sin guardia de seguridad.
- Trigger que rechaza asignar a una línea más del doble de buses que estaciones tiene.
- Trigger que genera automáticamente la alerta de refuerzo cuando una estación supera el 150 % de su aforo.
- Trigger que recalcula la distancia total de una línea a partir de la distancia entre sus estaciones.
- Procedimiento almacenado sp_cambiar_parqueo_bus, que cambia el parqueo de un bus en una sola transacción sin que quede sin parqueo.

Se eligió PostgreSQL frente a MySQL y SQL Server porque ofrece triggers y procedimientos en PL/pgSQL, tipos JSONB para guardar el trazo de las rutas, bloqueo de filas para evitar cobros duplicados y seguridad por fila (Row Level Security), sin costo de licencia. La base de datos se alojará en **Supabase**, un servicio que administra PostgreSQL en la nube e incluye conexión cifrada por SSL, respaldos diarios con siete días de retención y monitoreo. Como medida adicional se realizará un respaldo semanal con pg_dump almacenado fuera de Supabase.

Metodología de desarrollo: Scrum

El proyecto se desarrollará con **Scrum**, un marco de trabajo ágil que organiza el trabajo en ciclos cortos llamados sprints. Al final de cada sprint se entrega una parte funcional del sistema que se revisa con el Product Owner, lo que permite incorporar cambios en los requerimientos, como la información oficial de líneas y horarios o las reglas de la Tarjeta Ciudadana, sin rehacer el trabajo terminado.

Los roles de Scrum se asignarán de la siguiente manera:

- **Product Owner:** representante de la municipalidad, encargado de definir y priorizar los requerimientos a partir de los puntos del enunciado.
- **Scrum Master:** la consultora, responsable de organizar los sprints, eliminar impedimentos y velar por la calidad del proceso.
- **Equipo de desarrollo:** la consultora, encargada de analizar, programar, probar y documentar el sistema.

Los artefactos serán el **Product Backlog**, formado por los requerimientos funcionales y no funcionales; el **Sprint Backlog**, con las tareas seleccionadas para cada sprint; y el **Incremento**, que es la versión del sistema con todas sus pruebas aprobadas. En cada sprint se realizarán la planificación, el seguimiento diario, la revisión con demostración del sistema y la retrospectiva.

Una tarea se considerará terminada cuando su código cumpla el estándar de formato, tenga pruebas automatizadas aprobadas, esté relacionada con un requerimiento en la matriz de trazabilidad y su documentación esté actualizada. El proyecto se organizará en seis sprints de una semana:

Tabla 11.1. Planificación de sprints del proyecto.

Sprint	Fechas	Objetivo del sprint
1	21 – 27 sep	Análisis de requerimientos, modelo entidad relación y script de la base de datos
2	28 sep – 4 oct	Base de datos con sus reglas, autenticación por roles, módulos de administración, panel del operador, Tarjeta Ciudadana, mapa de rutas y reportes principales
3	5 – 11 oct	Bitácora de auditoría, reportes de alertas, recaudación y personal, registro de viajes y análisis de calidad del código
4	12 – 18 oct	Publicación en la nube, pruebas de carga y de seguridad, y monitoreo de disponibilidad
5	19 – 25 oct	Plan de pruebas, criterios de aceptación, manual de usuario y capacitación
6	26 – 30 oct	Documento final, presentación ejecutiva, video, carta de aceptación y entrega

Lenguaje de programación: PHP 8.4

El lenguaje principal será **PHP 8.4**, un lenguaje del lado del servidor diseñado para aplicaciones web, con tipado estricto, enumeraciones y un amplio ecosistema de librerías. Esta versión tiene soporte activo hasta el 31 de diciembre de 2026 y actualizaciones de seguridad hasta el 31 de diciembre de 2028, lo que cubre el tiempo de desarrollo y los primeros años de operación del sistema.

PHP se complementará con otros lenguajes, cada uno en su capa:

- **SQL y PL/pgSQL:** estructura de la base de datos, triggers, procedimiento almacenado y consultas de los reportes.
- **JavaScript:** mapa de las líneas, editor del trazo de las rutas y actualización periódica del panel de la estación.
- **HTML5 y CSS:** estructura y diseño de las pantallas, con plantillas Blade y estilos de Tailwind CSS.

Framework: Laravel 13

Para construir la aplicación se utilizará **Laravel 13**, el framework de PHP más utilizado para aplicaciones web. Laravel 13 se publicó el 17 de marzo de 2026, recibe correcciones hasta el tercer trimestre de 2027 y actualizaciones de seguridad hasta el 17 de marzo de 2028. Laravel organiza la aplicación con el patrón **Modelo – Vista – Controlador (MVC)**:

- **Modelo:** representa las entidades del sistema (Línea, Estación, Bus, Tarjeta, Alerta, entre otras) y sus relaciones.
- **Vista:** presenta la información al usuario mediante plantillas Blade.
- **Controlador:** recibe las solicitudes del usuario, valida los datos y coordina la respuesta.

Cuando un operador registra información, la ruta verifica que haya iniciado sesión y que tenga el rol correcto, el controlador valida los datos, la capa de servicios aplica las reglas de negocio (por ejemplo, el cobro del pasaje con viajes gratis y saldo) y el modelo guarda el registro, momento en que la base de datos verifica sus propias reglas.

También se utilizará **Eloquent ORM**, el componente de persistencia de Laravel. Su función será comunicar la aplicación con PostgreSQL mediante clases y objetos de PHP, reduciendo la necesidad de escribir consultas SQL manualmente para las operaciones comunes; los reportes usarán consultas SQL preparadas para documentarlas en este informe. Otros componentes de Laravel que se aprovecharán son:

- Migraciones para crear y versionar la estructura de la base de datos.
- Middleware de roles para separar las funciones del administrador y del operador.
- Validación de formularios con mensajes en español.
- Cifrado de contraseñas con bcrypt, protección CSRF y cifrado AES-256 de los datos personales sensibles.
- Comandos Artisan propios para tareas como la simulación de la operación y la exportación de las consultas de los reportes.

En la capa de presentación se usará **Tailwind CSS 4** compilado con **Vite**, y **Leaflet** con mapas de **OpenStreetMap** para mostrar las líneas y estaciones del Transmetro.

IDE: Visual Studio Code

El entorno de desarrollo será **Visual Studio Code**, un editor gratuito y liviano en el que se crearán los modelos, vistas, controladores, servicios, migraciones y pruebas del proyecto. Visual Studio Code proporciona funciones como:

- Editor de código con resaltado de sintaxis para PHP, Blade, SQL y JavaScript.
- Autocompletado y detección de errores mediante extensiones (PHP Intelephense, Laravel y Tailwind CSS IntelliSense).
- Terminal integrada para ejecutar Composer, Artisan, npm y las pruebas.
- Integración con Git y GitHub para el control de versiones.
- Conexión a PostgreSQL mediante extensiones de bases de datos.

Como herramientas complementarias se utilizarán **DBeaver** para diseñar y consultar la base de datos y generar el diagrama entidad relación, **Git y GitHub** para el control de versiones y el repositorio del proyecto, y **PlantUML** para los diagramas UML.

Plataforma de despliegue: Render y Supabase

La aplicación se publicará en **Render**, una plataforma en la nube que ejecutará el sistema dentro de un contenedor Docker con certificado HTTPS. Se utilizará el plan Starter, que mantiene la aplicación activa de forma permanente. La base de datos se alojará en **Supabase** con el plan Pro, que evita las pausas por inactividad e incluye respaldos diarios.

Cada PC de estación se conectará al sistema por Internet mediante HTTPS, por lo que no será necesario construir una red privada entre estaciones; una estación nueva solo necesitará un navegador y conexión a Internet. Esta arquitectura responde al punto 17 del enunciado, que indica que aún no se ha definido la mejor forma de conectar las máquinas.

Herramientas de calidad

Para asegurar la calidad del software durante todo el proyecto se utilizarán las siguientes herramientas:

- **Pest:** pruebas automatizadas unitarias y de funcionalidad que verifican cada regla del enunciado.
- **Laravel Pint:** revisión del formato del código según el estándar PSR-12.
- **GitHub Actions:** integración continua; en cada cambio revisa el formato y ejecuta todas las pruebas sobre una base PostgreSQL limpia antes de publicar.
- **SonarQube Cloud:** análisis estático para detectar errores, vulnerabilidades, código duplicado y deuda técnica.
- **k6:** pruebas de carga para medir el tiempo de respuesta con varios usuarios simultáneos.
- **UptimeRobot:** monitoreo de la disponibilidad del sistema en producción.

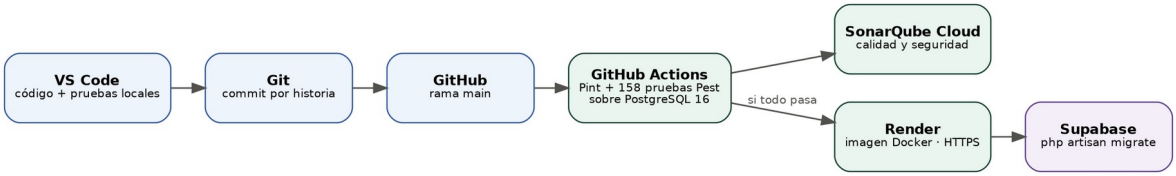


Figura 10. Flujo de integración y despliegue continuo.

Integración del stack tecnológico

El usuario accederá al sistema desde un navegador web en la PC de su estación o de la municipalidad. La solicitud viajará cifrada por HTTPS hasta la aplicación desarrollada con Laravel y PHP en Render. Las rutas y el middleware verificarán la sesión y el rol, los controladores validarán los datos y los servicios aplicarán las reglas de negocio. Posteriormente, Eloquent realizará las consultas o modificaciones en PostgreSQL, donde los triggers y restricciones aplicarán las reglas de integridad y generarán las alertas automáticas. La respuesta regresará al navegador como una página construida con Blade y Tailwind CSS.

El desarrollo se realizará en Visual Studio Code, las actividades se organizarán mediante Scrum, cada cambio será verificado por GitHub Actions y, finalmente, la aplicación se publicará en Render con la base de datos en Supabase.

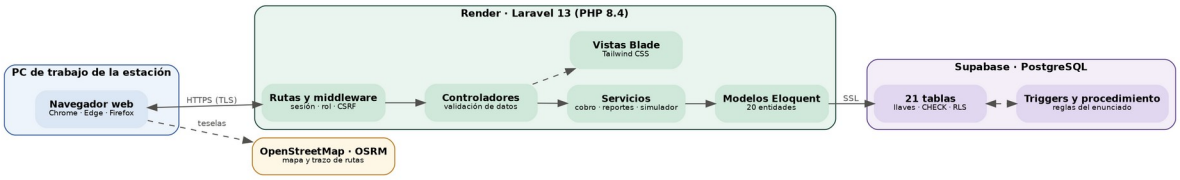


Figura 11. Integración de los componentes del stack durante una solicitud.

Tabla 11.2. Resumen del stack tecnológico.

Componente	Tecnología	Versión	Función en el proyecto	Costo
Base de datos	PostgreSQL en Supabase Pro	15 o superior	Almacenar la información y aplicar las reglas de integridad	US$25 al mes
Metodología	Scrum	—	Organizar el trabajo en sprints de una semana	Sin costo
Lenguaje	PHP	8.4	Lógica del servidor	Sin costo
Framework	Laravel	13	Estructura MVC, seguridad y acceso a datos	Sin costo
IDE	Visual Studio Code	Estable	Desarrollo del código	Sin costo
Gestor de BD	DBeaver Community	26.2	Diseño y consulta de la base de datos	Sin costo
Interfaz	Blade, Tailwind CSS, Leaflet	— / 4 / 1.9	Pantallas y mapa de rutas	Sin costo
Control de versiones	Git y GitHub	—	Repositorio e integración continua	Sin costo
Calidad	Pest, Pint, SonarQube Cloud, k6	—	Pruebas, formato, análisis y carga	Sin costo
Despliegue	Render Starter	—	Publicación de la aplicación con HTTPS	US$7 al mes

Tabla 11.3. Alternativas evaluadas y motivo de la selección.

Componente	Seleccionado	Alternativas evaluadas	Motivo de la selección
Base de datos	PostgreSQL	MySQL, SQL Server	Reglas en la base de datos con PL/pgSQL, JSONB y seguridad por fila, sin licencia y con hospedaje administrado accesible
Metodología	Scrum	Kanban, Cascada	Entregas frecuentes para revisión y adaptación a cambios en los requerimientos
Lenguaje y framework	PHP con Laravel	Python con Django, Java con Spring Boot	Mayor productividad para aplicaciones web, seguridad incluida y dominio previo del equipo
IDE	Visual Studio Code	PhpStorm, NetBeans	Gratuito, liviano y con extensiones para todo el stack
Despliegue	Render y Supabase	Servidor propio, Azure, AWS	Sin administración de servidores, costo mensual bajo y HTTPS incluido

12. MÉTRICAS DE SOFTWARE

En el Sistema de Control Transmetro se utilizarán métricas de proceso, de proyecto y de producto, evaluadas en cuatro dimensiones: eficacia, velocidad, calidad e impacto. Estas métricas se aplicarán durante todo el ciclo de vida del sistema: en la planificación, en cada sprint, en las pruebas finales y durante la operación. Permiten conocer el avance del proyecto, detectar problemas a tiempo, comprobar el cumplimiento de los requerimientos y demostrar el beneficio que el sistema genera para la municipalidad.

Objetivo general

Definir y evaluar indicadores que permitan medir la eficacia, la velocidad, la calidad y el impacto del Sistema de Control Transmetro, desde su desarrollo hasta su operación.

Objetivos específicos

- Medir la eficacia del proceso de desarrollo y el grado de cumplimiento de los requerimientos del enunciado.
- Controlar la velocidad de entrega del equipo y el cumplimiento del cronograma y del presupuesto.
- Evaluar la calidad del producto mediante su tamaño, complejidad, defectos, rendimiento, confiabilidad, seguridad y usabilidad.
- Determinar el impacto del sistema en la operación del Transmetro y en la gestión de la municipalidad.
- Establecer metas, herramientas y responsables para la recolección de cada métrica.

Marco conceptual

Una **medida** es un valor cuantitativo de un atributo (por ejemplo, el número de defectos); una **métrica** relaciona medidas para obtener información (defectos por cada mil líneas de código); y un **indicador** es una métrica con una meta que permite tomar decisiones. Según Pressman, las métricas de software se agrupan en tres tipos según lo que miden:

- **Métricas de proceso:** miden la forma en que se desarrolla el software para mejorar el proceso en este y en futuros proyectos. Son estratégicas y de largo plazo; por ejemplo, la eficiencia de remoción de defectos o el tiempo de ciclo de una tarea.
- **Métricas de proyecto:** permiten al equipo controlar el proyecto en curso: avance, esfuerzo, costo, cronograma y riesgos. Son tácticas; por ejemplo, la velocidad del sprint o el índice de desempeño del cronograma.

- **Métricas de producto:** miden las características del software entregado: tamaño, complejidad, calidad, rendimiento, confiabilidad, seguridad y facilidad de uso, alineadas con el estándar ISO/IEC 25010.

Cada métrica se evalúa además en una de cuatro dimensiones, que responden a las preguntas que la municipalidad y el equipo necesitan contestar:

Tabla 12.1. Dimensiones de evaluación de las métricas.

Dimensión	Pregunta que responde	Ejemplos de métricas
Eficacia	¿Se logra lo que se planificó y lo que pide el enunciado?	Requerimientos cumplidos, avance funcional, cumplimiento del sprint, eficiencia de remoción de defectos
Velocidad	¿Qué tan rápido se entrega y qué tan rápido responde el sistema?	Velocidad del equipo, tiempo de ciclo, frecuencia de despliegue, tiempo de respuesta
Calidad	¿El software es correcto, mantenible, seguro y confiable?	Densidad de defectos, cobertura de pruebas, complejidad, vulnerabilidades, disponibilidad
Impacto	¿Qué beneficio genera el sistema para la municipalidad y los usuarios?	Tiempo de atención de alertas, horas administrativas ahorradas, satisfacción de usuarios, VAN

Para seleccionar las métricas se usó el enfoque **GQM (Objetivo – Pregunta – Métrica)**: cada métrica responde a una pregunta derivada de un objetivo del proyecto. Las metas se clasifican con un semáforo: **verde** cuando se cumple la meta, **amarillo** cuando el valor está a menos del 10 % de la meta y requiere una acción de mejora en el siguiente sprint, y **rojo** cuando no se cumple y debe corregirse antes de cerrar el sprint.

Métricas de proceso

Evalúan cómo trabaja el equipo con Scrum y con el flujo de integración continua. Se basan en las prácticas de medición de entrega de software de DORA (frecuencia de despliegue, tiempo de entrega de cambios, tasa de fallos y tiempo de recuperación) y en las métricas de calidad del proceso de Pressman.

Tabla 12.2. Métricas de proceso.

Métrica	Dimensión	Fórmula o forma de medición	Meta	Herramienta
Eficiencia de remoción de defectos (DRE)	Calidad	DRE = E ÷ (E + D), donde E son los defectos encontrados antes de la entrega y D los encontrados después	≥ 95 %	Issues de GitHub
Eficacia de las pruebas automatizadas	Eficacia	Defectos detectados por pruebas ÷ defectos totales × 100	≥ 70 %	Issues de GitHub, Pest
Cumplimiento de la definición de terminado	Eficacia	Tareas que cumplen formato, pruebas, trazabilidad y documentación ÷ tareas terminadas × 100	100 %	GitHub Actions, tablero Scrum
Tiempo de ciclo	Velocidad	Días desde que una tarea inicia hasta que está terminada	≤ 3 días	Tablero Scrum
Tiempo de entrega de cambios	Velocidad	Horas desde el commit hasta que el cambio está en producción	≤ 1 día	GitHub, Render
Frecuencia de despliegue	Velocidad	Publicaciones en producción por semana	≥ 1 por sprint	Render
Tasa de fallo de cambios	Calidad	Despliegues que causan fallas ÷ despliegues totales × 100	≤ 15 %	Render, Issues
Tiempo de recuperación de un despliegue fallido	Velocidad	Tiempo hasta restaurar el servicio tras una falla	< 1 hora	Render, UptimeRobot
Porcentaje de retrabajo	Calidad	Horas dedicadas a corregir defectos ÷ horas totales × 100	≤ 10 %	Registro de horas
Productividad	Velocidad	Puntos de función terminados ÷ horas hombre	≥ 0.17 PF por hora (6 h por PF)	Registro de horas

Métricas de proyecto

Permiten controlar el proyecto frente a la línea base de la propuesta monetaria: 418.46 puntos de función ajustados, 2,510.76 horas hombre, 7.47 meses con dos desarrolladores y un costo de desarrollo de Q99,010.63. El avance se controlará con la técnica de **valor ganado**, usando como unidad los puntos de función terminados.

Tabla 12.3. Métricas de proyecto.

Métrica	Dimensión	Fórmula o forma de medición	Meta	Herramienta
Avance funcional	Eficacia	PF terminados ÷ PF totales (418.46) × 100	100 % al cierre	Matriz de trazabilidad
Cumplimiento de requerimientos	Eficacia	Requerimientos RF y RNF aceptados ÷ requerimientos totales × 100	100 %	Matriz de trazabilidad
Cumplimiento del sprint	Eficacia	Tareas terminadas ÷ tareas comprometidas en el sprint × 100	≥ 90 %	Tablero Scrum
Velocidad del equipo	Velocidad	Puntos de función terminados por sprint	≈ 70 PF por sprint (418.46 ÷ 6)	Tablero Scrum
Gráfico de trabajo pendiente (burndown)	Velocidad	PF pendientes al final de cada día del sprint	Llegar a 0 al cerrar el sprint	Tablero Scrum
Índice de desempeño del cronograma (SPI)	Velocidad	SPI = valor ganado ÷ valor planificado	≥ 0.95	Hoja de control
Índice de desempeño del costo (CPI)	Eficacia	CPI = valor ganado ÷ costo real	≥ 0.95	Hoja de control
Desviación del esfuerzo	Eficacia	(Horas reales − horas estimadas) ÷ horas estimadas × 100	≤ ±10 %	Registro de horas
Volatilidad de requerimientos	Calidad	Requerimientos agregados o modificados ÷ requerimientos totales × 100	≤ 15 %	Product Backlog
Riesgos mitigados	Eficacia	Riesgos con plan de mitigación aplicado ÷ riesgos identificados × 100	100 % de riesgos altos	Matriz de riesgos
Rentabilidad del proyecto	Impacto	Valor actual neto a 5 años al 10 % y relación beneficio/costo	VAN > 0; B/C > 1	Estudio de factibilidad

Métricas de producto

Evalúan el software que se entregará a la municipalidad. Incluyen las métricas de calidad propuestas por Pressman (corrección, facilidad de mantenimiento, integridad y facilidad de uso) y las características del estándar ISO/IEC 25010.

Tamaño y complejidad

Tabla 12.4. Métricas de tamaño y complejidad.

Métrica	Dimensión	Fórmula o forma de medición	Meta	Herramienta
Puntos de función ajustados	Eficacia	PFA = PFSA × (0.65 + 0.01 × Σ factores de ajuste)	418.46 PFA	Hoja de estimación FPA
Líneas de código	Calidad	Líneas de código fuente sin comentarios ni líneas en blanco, por capa	Registrar por sprint	SonarQube Cloud
Complejidad ciclomática	Calidad	CC = 1 + número de decisiones de un método	Promedio ≤ 5; ningún método > 10	SonarQube Cloud
Complejidad cognitiva	Calidad	Dificultad para entender un método según anidamiento y saltos	≤ 15 por método	SonarQube Cloud
Código duplicado	Calidad	Líneas duplicadas ÷ líneas totales × 100	< 3 %	SonarQube Cloud
Deuda técnica	Calidad	Tiempo para corregir problemas de calidad ÷ tiempo de desarrollo × 100	Calificación A (< 5 %)	SonarQube Cloud

Calidad del producto

Tabla 12.5. Métricas de calidad del producto.

Métrica	Dimensión	Fórmula o forma de medición	Meta	Herramienta
Corrección	Calidad	Defectos encontrados ÷ miles de líneas de código (KLOC)	≤ 2 por KLOC	Issues de GitHub
Cobertura de pruebas	Calidad	Líneas ejecutadas por las pruebas ÷ líneas totales × 100	≥ 80 % en servicios y modelos	Pest con PCOV
Pruebas aprobadas	Eficacia	Pruebas aprobadas ÷ pruebas ejecutadas × 100	100 % en cada publicación	Pest, GitHub Actions
Reglas del enunciado verificadas	Eficacia	Reglas con prueba automatizada ÷ reglas del enunciado × 100	100 %	Pest
Facilidad de mantenimiento (MTTC)	Velocidad	Tiempo medio para analizar, implementar, probar y publicar un cambio	≤ 2 días	Issues de GitHub
Integridad	Calidad	Integridad = Σ [1 − (amenaza × (1 − seguridad))] por tipo de ataque	≥ 0.95	Análisis de riesgos de seguridad
Vulnerabilidades	Calidad	Vulnerabilidades del código y de las dependencias por severidad	0 críticas y 0 altas	SonarQube Cloud, composer audit
Protección de datos	Calidad	Tablas con Row Level Security y campos sensibles cifrados ÷ totales × 100	100 %	Catálogo de PostgreSQL

Rendimiento, confiabilidad y facilidad de uso

Tabla 12.6. Métricas de rendimiento, confiabilidad y usabilidad.

Métrica	Dimensión	Fórmula o forma de medición	Meta	Herramienta
Tiempo de respuesta (percentil 95)	Velocidad	Tiempo bajo el que se atiende el 95 % de las solicitudes	< 2 s con 50 usuarios simultáneos	k6
Capacidad de procesamiento	Velocidad	Solicitudes atendidas por segundo	≥ 20 por segundo	k6
Tiempo de generación de alertas	Velocidad	Tiempo entre la actualización del aforo y la creación de la alerta	Misma transacción (< 1 s)	Pruebas automatizadas
Disponibilidad	Calidad	Tiempo en servicio ÷ horario de servicio (04:30 a 21:00) × 100	≥ 99 %	UptimeRobot
Tiempo medio entre fallas (MTBF)	Calidad	Horas de operación ÷ número de fallas	≥ 720 horas	UptimeRobot
Tiempo medio de recuperación (MTTR)	Velocidad	Tiempo de interrupción ÷ número de fallas	< 1 hora	UptimeRobot, Render
Tasa de errores del servidor	Calidad	Respuestas con error 5xx ÷ solicitudes totales × 100	< 1 %	Registros de Render
Facilidad de uso (SUS)	Impacto	Cuestionario de usabilidad de 10 preguntas, puntaje de 0 a 100	≥ 70 puntos	Cuestionario SUS
Tasa de éxito de tareas	Eficacia	Tareas completadas sin ayuda ÷ tareas intentadas × 100	≥ 90 %	Prueba con operadores
Tiempo por tarea	Velocidad	Tiempo para registrar un pase o actualizar el aforo	< 30 segundos	Prueba con operadores

Métricas de impacto

Miden el beneficio que el sistema aporta a la operación del Transmetro y a la gestión de la municipalidad. Se obtendrán de los reportes del sistema y se compararán con la situación anterior a su implementación.

Tabla 12.7. Métricas de impacto.

Métrica	Dimensión	Fórmula o forma de medición	Meta	Herramienta
Tiempo de atención de alertas de refuerzo	Impacto	Promedio entre la creación y el cierre de la alerta	< 10 minutos	Reporte de alertas
Horas de saturación por estación	Impacto	Horas al mes con aforo ≥ 150 %	Reducción mensual	Reporte de estaciones
Buses con espera extra	Impacto	Visitas con ocupación < 25 % ÷ visitas totales × 100	Reducción mensual	Reporte de visitas
Cumplimiento de flota por línea	Impacto	Líneas con buses entre n y 2n ÷ líneas totales × 100	100 %	Reporte de líneas
Horas administrativas ahorradas	Impacto	Horas de consolidación manual antes − después	≥ 1,440 horas al año	Encuesta a la municipalidad
Recaudación conciliada	Impacto	Monto conciliado ÷ monto registrado en molinetes × 100	100 %	Reporte de recaudación
Adopción del sistema	Impacto	Operadores que usan el sistema a diario ÷ operadores capacitados × 100	≥ 90 %	Bitácora de auditoría
Valor actual neto (VAN)	Impacto	Σ flujos netos descontados al 10 % − inversión	> 0 (estimado Q65,897 a 5 años)	Estudio de factibilidad
Relación beneficio/costo	Impacto	Beneficios descontados ÷ costos descontados	> 1 (estimado 1.38)	Estudio de factibilidad
Período de recuperación	Impacto	Inversión ÷ flujo neto anual	< 3 años (estimado 2.5)	Estudio de factibilidad

Evaluación de las métricas

La siguiente tabla resume los indicadores clave de cada dimensión, su meta para el proyecto completo y su evaluación con la medición realizada sobre el sistema construido. Los indicadores de operación e impacto se evaluarán una vez que el sistema esté en producción.

Tabla 12.8. Evaluación de los indicadores clave por dimensión.

Dimensión	Indicador	Meta	Resultado medido	Evaluación
Eficacia	Reglas del enunciado verificadas	100 %	100 % (9 de 9 reglas con prueba)	Verde
Eficacia	Pruebas aprobadas	100 %	100 % (158 de 158)	Verde
Eficacia	Avance funcional	100 % al cierre	93.3 % (390.40 de 418.46 PFA)	En seguimiento
Eficacia	Eficiencia de remoción de defectos	≥ 95 %	100 % (9 defectos corregidos antes de la entrega)	Verde
Velocidad	Velocidad del equipo	≈ 70 PF por sprint	≈ 195 PF por sprint en los dos primeros sprints	Verde
Velocidad	Ejecución de pruebas en integración continua	< 60 segundos	7.3 segundos	Verde
Velocidad	Tiempo de respuesta (p95)	< 2 s	Se medirá con k6 en el sprint 4	Pendiente
Calidad	Corrección (densidad de defectos)	≤ 2 por KLOC	1.55 por KLOC	Verde
Calidad	Complejidad ciclomática promedio	≤ 5	1.98	Verde
Calidad	Métodos con complejidad mayor a 10	0	3 (refactorización en el sprint 3)	Amarillo
Calidad	Tablas con Row Level Security	100 %	100 % (21 de 21)	Verde
Calidad	Cobertura de pruebas	≥ 80 %	Se medirá con PCOV en el sprint 3	Pendiente
Impacto	Relación beneficio/costo	> 1	1.38 (estimado en la factibilidad)	Verde
Impacto	Tiempo de atención de alertas	< 10 minutos	Se medirá en producción	Pendiente
Impacto	Satisfacción de los operadores (SUS)	≥ 70	Se medirá en la capacitación	Pendiente

Plan de recolección de métricas

Tabla 12.9. Momento, frecuencia y responsable de la medición.

Tipo de métrica	Momento	Frecuencia	Responsable
Proceso	Durante el desarrollo y en cada publicación	En cada cambio y al cerrar el sprint	Scrum Master
Proyecto	Planificación y desarrollo	Diaria (burndown) y en cada revisión de sprint	Product Owner y Scrum Master
Producto: tamaño, complejidad y calidad	Desarrollo y pruebas finales	En cada cambio	Equipo de desarrollo
Producto: rendimiento y confiabilidad	Pruebas finales y producción	Antes de cada versión y de forma continua	Equipo de desarrollo y soporte
Producto: facilidad de uso	Capacitación y prueba piloto	Una vez por versión	Scrum Master
Impacto	Operación	Mensual	Municipalidad

Resultados esperados

Con la aplicación de estas métricas se espera entregar un sistema **eficaz**, que cumpla el 100 % de los requerimientos y reglas del enunciado con pruebas automatizadas; **rápido**, tanto en la entrega de cambios como en el tiempo de respuesta para los operadores; de **calidad**, con baja complejidad, menos de dos defectos por cada mil líneas de código, sin

vulnerabilidades críticas y con una disponibilidad de al menos 99 %; y con un **impacto** positivo y medible para la municipalidad, reflejado en una atención más rápida de las estaciones saturadas, el control de la flota y la recaudación, y una inversión que se recupera en menos de tres años.